Kaggle Competition: Facial Feature Classification

ENGE707 Data Engineering and AI

Shawn Lee (23204035)

Auckland University of Technology

Table of Content

1. Dataset Exploration and Understanding **3**

1.1 Problem 3

1.2 Data Inspection 3

1.3 Number of Instances in each class 4

1.4 Class Image Inspection and Evaluation 5

1.5 Explanation of usage of Training, Validation and Kaggle Submission 6

2. Data Preprocessing and Pipeline Preparation **7**

2.1 Data Preparation 7

2.2 Training and Validation Split 8

2.3 Resize Image 9

2.4 Scale and Normalisation of Pixel 10

2.5 Encoding 10

2.6 Data Augmentation 11

2.7 Data Loaders 12

2.8 Confirm preprocessing Consistency and Leakage prevention 12

3. Model Selection and Pipeline Development **13**

3.1 Model Selection 13

3.2 Model Architecture 14

3.3 Transfer Learning and Model Modification 15

3.4 Model Input and Output Configuration 15

3.5 Training Hyperparameters 16

3.6 Training Pipeline Implementation 17

3.7 Kaggle Test Prediction Pipeline 17

4. Training Strategy and Overfitting/Underfitting Management **17**

5. Model Evaluation and Kaggle Submission **17**

6. Results Analysis and Improvement Strategies **18**

7. Report Writing and Reproducibility **18**

References 18

1. Dataset Exploration and Understanding

1.1 Problem



The problem focuses on distinguishing between cats and dogs and which animal facial features (eye, nose, mouth or face) an image shows. This is a classification problem where there are 8 different classes: CatEyes, DogEyes, CatFace, DogFace, CatMouth, DogMouth, CatNose, or DogNose that the image can be classified as. If the model can succeed in providing a high accuracy and other metrics in the prediction results, the model could be used as the first step to further classify the breed and visual anomalies of the cat / dog. This prediction would allow for example a system to automatically describe what section of the animal is being checked and check the region for possible anomalies.

1.2 Data Inspection

```
Train rows, columns: (63660, 2)
Column dtypes: {'image_id': 'str', 'class': 'str'}
Null values: {'image_id': 0, 'class': 0}
Duplicate ids in CSV: 0
Train id range: 1 - 63660
Train file types: {'.jpg': 63660}

Test rows, columns: (29443, 2)
Duplicate ids in sample_submission: 0
Test id range: 1 - 29443
Test file types: {'.jpg': 29443}
Test classes in sample_submission: {'CatEye': 4567, 'DogEye': 4221, 'CatFace': 3571, 'CatMouth': 3570, 'CatNose': 3457, 'DogFace': 3407, 'DogMouth': 3341, 'DogNose': 3309}

Train files: 63660 | Test files: 29443
```

Figure 1.2.1 : Dataset Structure

The Kaggle competition provided testing and training folders as well as included the train_labels.csv which contains the mapping of the “image_id” and “class” columns, the “image_id” 00001 links to 00001.jpg in the train folder and the class is the target. The test folder contains the unlabelled testing images and the train folder contains the labelled training images (connected to the train_labels.csv).

In the training dataset, there are 63,660 rows and in the testing dataset there are 29,443 rows. The train_labels.csv consists of 2 columns “image_id” and “class”. With “image_id” as the unique identification of the image and class being the target.
In Figure 1.21, it showed that there are 0 duplicate ids in CSV and the id correctly ranges from 1 to 63660 which is the correct amount of images in the training folder.

The sample_submission.csv shows what the structure of the data submission to kaggle would need to be like with an unpadded “image_id” (e.g 00001 to 1) and the class column that describes the predicted target attribute.

1.3 Number of Instances in each class

	class	instances
0	CatEye	13884
1	CatFace	6942
2	CatMouth	6942
3	CatNose	6942
4	DogEye	11580
5	DogFace	5790
6	DogMouth	5790
7	DogNose	5790

Figure 1.3.1 : Class Distribution

In the train_labels.csv there, it shows that not all classes are distributed evenly. With CatEye (13,884) being the dominant class having more than twice the amount of instances than the other classes such as DogFace, DogMouth, and DogNose (5,790). This shows that there is a class imbalance in the training dataset. This would be considered during the preprocessed stage where the training and validation split.

1.4 Class Image Inspection and Evaluation



Figure 1.4.1 : Example Classification Images

The images shown of each class (CatEye, CatFace, CatMouth, CatNose, DogEye, DogFace, DogMouth, and DogNose) all showed varying infomation. The images shown appeared to vary in resolution with some seemingly more pixelated than others, Some images do not clearly show the features such as the DogFace id17364 where it is hard to view the dog's face due to the fur color and clarity.

Additionally some of the images shown contain other classes that they weren't supposed to capture such as the DogNose id 27348 showed the Nose as well and 1 eye. This would make classification for models much more difficult due to the combination of multiple target classes.

The lighting and position of the images are also different as some appear overly bright and some appear dark. The head position of the cats and dog images are also not perfectly horizontal (e.g the CatNose Id 62280).

Image dimension summary:

	width			height		
	min	median	max	min	median	max
split						
test	8	82.0	1832	9	80.0	1978
train	4	84.0	2219	4	80.0	2219

Figure 1.4.2 : Image Dimension Summary

Further confirming the dimension of the images, Figure 1.4.2 shows the summary of all the images, with the train images ranging from 4px to 2219px and the height ranging from 4px to 2219px. This range is apparent for the test images as well.

```
File-reading or decoding failures: 0
```

Figure 1.4.3 : Checking Corrupted Files

Confirmed that there are no corrupted files in both the testing image folder and the training image folder.

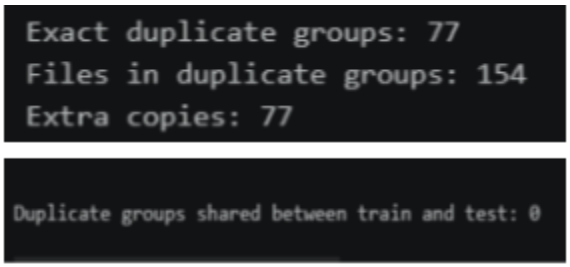


Figure 1.4.4 : Checking Duplicates

There are 77 exact duplicate groups from using SHA 256 hash in both the testing folder and the training folder. This means that there are 77 different groups of images, and 2 images have the same hash. Though there are duplicates in both the testing and training folder, there are no shared duplicates between those folders. Therefore to prevent data leakage, the duplicates would need to be kept in the same dataset for training or validation. This checks only if the byte content is the same, it does not check if the image was resized, or recompressed and each image has not been checked visually if they are similar. Further evaluation would proceed in the preprocessing stage whether duplicates would need to be removed.

1.5 Explanation of usage of Training, Validation and Kaggle Submission

For the dataset, the test dataset (images) would not be used in training in any way, only used for the testing for the final kaggle submission. The training dataset (images) would be applied to the 80 / 20 split so that 80% of the data would be for training and the 20% would be for validation. Stratified splitting would be applied to ensure that the class proportion is the same for both the training and the validation subsets. The 80% training data would be used to actually learn the pattern from images to adjust appropriate weights for each classification and the validation dataset is used to evaluate the performance on unseen data, so that further improvement could be made. Once the performance is satisfactory the trained deep learning model would be used on the test dataset to predict all instances in the dataset, the result of the prediction would be saved in a csv file to be submitted to kaggle.

2. Data Preprocessing and Pipeline Preparation

In the preprocessing stage I have decided to use a pretrained CNN (Convolutional Neural Networks) model called ResNet18. This preprocessing pipeline would be based on trying to optimize the images so it would perform the best on the defined model. Further explanation on why I chose this model and architecture would be in section 3 (Mode selection and pipeline development).

2.1 Data Preparation

To prepare the data for the other preprocessing stage (training-validation split, image resize, etc.). The training dataset ensured that the train_label.csv file had an appropriate mapping between each image_id and its image path.

```
Training duplicate groups: 15
Files in duplicate groups: 30
Extra duplicate copies: 15
```

Figure 2.1.1 : Duplicates in training dataset

```
Duplicate groups with conflicting labels: 0
Empty DataFrame
Columns: [image_id, class, path, sha256]
Index: []
```

Figure 2.1.2 : Checking Duplicates conflicting labels

In Task 1, it was identified that there are 77 exact duplicate groups using hash; therefore, another check has been performed in the training dataset to see the number of duplicate groups. The SHA-256 identified that there are 15 duplicate groups, and no conflicting labels were found; therefore, the images are duplicates. The duplicate images are removed from the training dataset because splitting the dataset into two 2 subsets, training and validation, could potentially have the same image appear on both the training and validation sets. This would cause data leakage, where they memorise the images and provide a falsely high validation accuracy. The duplicates were removed from the training dataframe, and the total training dataset went from 63,660 to 63,645 images.

```
Before: 63660
After: 63645
Removed: 15
```

Figure 2.1.3 : Summary of training dataset after removal

2.2 Training and Validation Split

```
Training images: 50916
Validation images: 12729
Total: 63645
```

class	Original (%)	Training (%)	Validation (%)
CatEye	21.80	21.80	21.80
CatFace	10.91	10.91	10.91
CatMouth	10.91	10.91	10.90
CatNose	10.91	10.91	10.90
DogEye	18.19	18.19	18.19
DogFace	9.09	9.09	9.10
DogMouth	9.10	9.10	9.10
DogNose	9.10	9.10	9.10

```
No overlapping image IDs or exact duplicate hashes.
```

Figure 2.2.1 : 80/20 split Confirmation

The 80/20 split method was used to split 80% to the training subset (50,916 images) and 20% to the validation subset (12,729 images). This allows the model to train on the majority of the total dataset and have enough unseen data to validate. To maintain the same amount of proportions across both subdatasets, stratification is used to ensure that the class proportions are approximately similar relative to the proportions of the original dataset. This is to prevent overrepresentation of a class in one dataset rather than the other from the original dataset. A seed of 42 is used to ensure this split is reproducible.

2.3 Resize Image



Figure 2.3.1 : 6 Examples on resizing applied

From the investigation of the dataset, we can see that the images vary in size, whereas the pretrained CNN model uses a standard image size (224 x 224). Therefore, images are resized proportionally to 224 x 224 if possible, and padding is applied to fill up the image to the set standard image size; this is to ensure that the original aspect ratio is preserved using padding, unlike resize methods such as cropping, which may remove important facial features. For larger images, it is scaled down proportionally, and padding is added to reach the set size stated.

For smaller images, upscaling the image generally preserves details; scaling down for larger images may remove finer image details but is necessary to match the set image dimensions. The main benefit for this type of resizing is that the image framing is maintained and does not introduce distortion of the image.

In Figure 2.3.1, the first image padding is applied to the image because the image can't scale properly to fit 224x224; therefore, black bars are added to the top and bottom, and the image does not show any distortion.

2.4 Scale and Normalisation of Pixel

```
Image class: CatNose
Original image size: (224, 224)

After rescaling:
Tensor shape: torch.Size([3, 224, 224])
Minimum value: 0.0
Maximum value: 1.0

After normalisation:
Tensor shape: torch.Size([3, 224, 224])
Minimum value: -2.1179039478302
Maximum value: 2.5354249477386475
```

Figure 2.4.1 : A example of scaling and normalisation

The image pixel values first consist of values from 0- 255, as they follow the RGB scale. It is rescaled to 0 - 1 to ensure that the image pixel values are compatible with the CNN model, as the input values are on a more consistent numerical scale than the original pixel values.

After scaling the pixel values are then normalised because the ResNet18 model follows a set standard deviation and mean. The standard deviation for red is 0.229, green 0.224, and 0.225 for blue. The mean is 0.485 for red, 0.456 for green, and 0.406 for blue. These values are used to normalize since the pretrained ResNet18 model was trained using these preprocessing values. Figure 2.4.1 shows an example of an image being scaled to 0-1 and normalised to a value range of -2.11 to 2.53.

2.5 Encoding

```
Class mapping: {'CatEye': 0, 'CatFace': 1, 'CatMouth': 2, 'CatNose': 3, 'DogEye': 4, 'DogFace': 5, 'DogMouth': 6, 'DogNose': 7}

Training examples:
  image_id     class  label
0    26977   CatNose      3
1    43790  DogMouth      6
2    60045    CatEye      0
3    47430    DogEye      4
4    50148  DogMouth      6
```

Figure 2.5.1 : Integer Encoding Evidence

Encoding is used to turn text labels into numbers in order for the model to map the classes easily. Integer encoding is utilized since the 8 different classes can be represented with unique values, and these values do not need to be represented in order. This method selects a unique identifier value to assign to a class; in Figure 2.5.1, CatEye is 0, CatFace is 1, CatMouth is 2, CatNose is 3, DogEye is 4, DogFace is 5, DogMouth is 6, and DogNose is 7.

(Note: For submission, the predicted labels will be converted back to their corresponding original labels for Kaggle submission.)

2.6 Data Augmentation

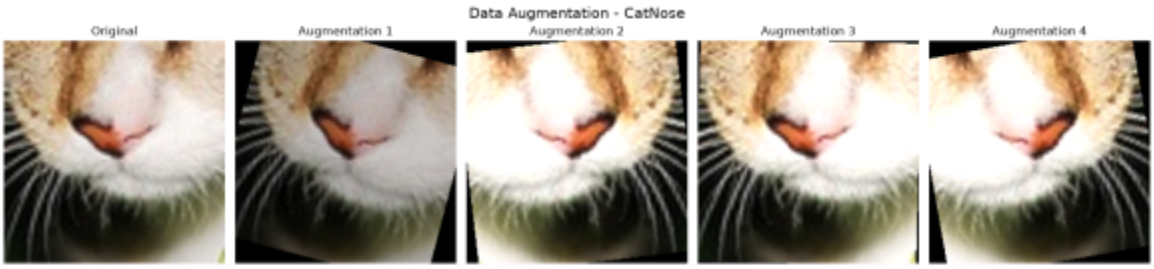


Figure 2.6.1: Example Augmentation

In the task, during the data exploration, it was concluded that there is much variation in the images, such as brightness, orientation of animal features, etc. This means that the model would recognize facial features under a wide variation of image conditions. Augmentation allows the model to improve generalization for unseen images and reduce overfitting. The data had been augmented by introducing horizontal flipping, rotations, and brightness adjustments; these augmentations are randomly applied to the training subset, and different variations of augmentation could be generated during training. In Figure 2.6.1, the first image shows the original, while the other images show the applied augmentations, with Augmentation 1 rotated to the right and lowered in brightness. Augmentation 4 shows the image rotated left, with increased brightness and horizontally flipped. This is not applied to the testing and validation datasets, as these datasets need to be kept unchanged from augmentation and only undergo other preprocessing stages, as the images need to be consistent to evaluate results.

2.7 Data Loaders

Data loaders are used in the preprocessing as this is to ensure that not all images are loaded at one time. This makes sure that the entire dataset isn't stored in the GPU memory. The batch size used is initially 32 at this stage as this is a good recommended starting point between efficiency and memory consumption (This could be changed later on depending on the memory usage, efficiency and validation results) . The images are shuffled in the training subset to allow the model to encounter different orders of images, for example the image could encounter thousands of the same class before others thus the weights would temporarily be more biased towards the most recently dominated class. While the training subset is shuffled the validation images would not be just to provide a predictable order and to provide an easier way to map predictions to their image_id.

```
Training samples: 50916
Validation samples: 12729
Training batches: 1592
Validation batches: 398

Test samples: 29443
Test batches: 921
```

Figure 2.7.1 : Number of batches of all datasets

The images are loaded individually before processing including resizing, augmentation, and normalisation and after grouped up to form batches.

2.8 Confirm preprocessing Consistency and Leakage prevention

```
Training preprocessing:
Compose(
    <__main__.ResizeAndPad object at 0x000001F7ACF8E8B0>
    RandomHorizontalFlip(p=0.5)
    RandomRotation(degrees=[-15.0, 15.0], interpolation=nearest, expand=False, fill=0)
    ColorJitter(brightness=(0.8, 1.2), contrast=None, saturation=None, hue=None)
    ToTensor()
    Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
)

Validation preprocessing:
Compose(
    <__main__.ResizeAndPad object at 0x000001F7ACF7EE70>
    ToTensor()
    Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
)

Test preprocessing:
Compose(
    <__main__.ResizeAndPad object at 0x000001F7ACF7EE70>
    ToTensor()
    Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
)
```

Figure 2.8.1 : Confirm preprocessing between validation, training and testing

The preprocessing stage is applied throughout the test dataset, the training dataset as well as the validation dataset, though both the test dataset and the validation dataset is excluded from augmentation. Figure 2.8.1 shows what transformations have been applied to each dataset. Other than augmentation, the same resizing, padding and normalisation methods are applied to all datasets to

ensure the processed images are suitable for the resNet18 model.

Throughout the preprocessing data leakage had been minimised by removing the duplicates images before the 80/20 stratified training validation split in Figure 2.1. This makes sure that the split would not contain the same hashed images in both the validation and training dataset. The test set was kept separated from the training labelled dataset and is only used during the kaggle submission to submit the final prediction. These techniques helped minimise data leakage but it does not guarantee that there are no visually similar images even if the file hashes are checked for duplicates.

3. Model Selection and Pipeline Development

3.1 Model Selection

Convolutional Neural Network (CNN) is selected as the deep learning model because it is capable of learning image patterns. These patterns include edges, textures, and shapes in a picture of a dog and a cat. Convolutional filters contain learnable weights that are applied to local areas of an image to detect specific visual patterns. As images go through multiple convolutional layers, extracted features become more and more complex, thus allowing more complex patterns to be recognized and eventually used to classify. The use of convolutional filters allows the identification of the current pixel with the adjacent pixels, allowing CNN to learn spatial relationships such as the arrangement of shapes and edges. Additionally, CNN uses less weights due to parameter sharing, thus generally lowering the total number of weights compared to a fully connected neural network. These characteristics are what the dataset needs, as shown in Figure 1.4.1, where the images show varying quality, facial orientation, etc.; therefore, the model would need to recognize similar patterns across images as well as learn local pixel arrangement to identify all 8 classes.

A Transfer learning model is used, called ResNet-18; this is a pretrained CNN model based on weights pretrained on the ImageNet dataset to identify 1,000 categories. This allows the previously learned

weights to be used on the training dataset. The final classification layer would be changed from 1000 categories to 8 target classes.

ResNet18 where it provides a pathway (residual connection) between convolutional layers, unlike a basic CNN that processes data through convolutional layers in order. This shortcut is important in making sure that the gradient reduces the vanishing gradient issue where the gradient becomes increasingly smaller as it propagates back through the convolutional layers. This results in slower learning for the model, as the initial convolutional layers would receive minor weight changes. The added pathway makes sure that gradients are able to propagate back more effectively (He et al., 2016).

Compared to other models, ResNet18 is relatively lightweight compared to other larger pretrained architectures like VGG16. According to Tanvir et al. (2026), ResNet18 contains approximately 11.18 million parameters compared to VGG16's 134.27 million parameters. The model would be trained on Kaggle's GPU environment therefore has limited resources and usage; therefore, selecting a lower-parameter model is a better choice for this project.

3.2 Model Architecture

The architecture of ResNet18 starts by passing the original image (224x224 pixels) through the initial convolutional layer, which includes 3 input channels (RGB) and 64 different filters, each with a 7x7 kernel. Once the feature map is produced, batch normalisation is applied, which basically normalises the feature activation values. ReLU activation then introduces non-linearity that replaces negatives with zeros and keeps the positive activation values. Max pooling is used after this process to reduce the spatial size of the feature map.

After that process, this is followed by 4 layers; each layer consists of 2 residual blocks, each residual block contains a convolutional layer, batch normalisation, ReLU, another convolutional layer, and

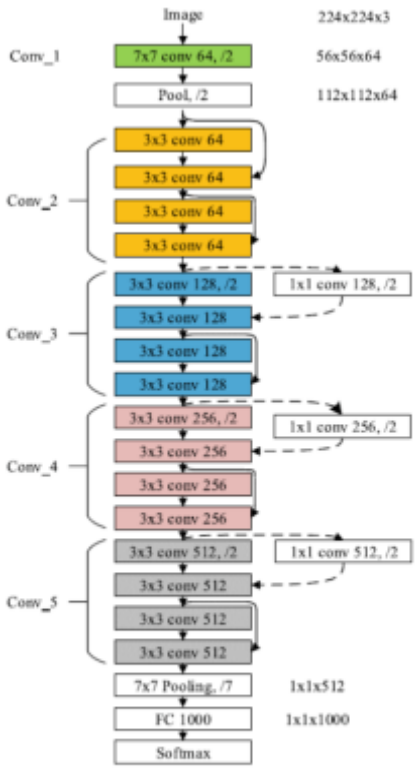


Figure 3.2.1 : ResNet18 Architecture
Al Rikabi and Muhammad (2024, Figure 2).

another batch normalisation. This residual block contains a path that skips the main convolutional layers from the input to the output feature map to add the initial input feature map. Each layer gets an increasingly higher number of feature channels (64, 128, 256 to 512) so that it can learn more complex patterns.

Once these feature maps are produced from those 4 layers, the final global average pooling is applied to the 512 feature maps to find the mean value of each. This is then flattened into a single vector and transferred to the classification layer. The fully connected classification layer converts the 512 feature values into 8 output values relating to the 8 classes in the dataset.

3.3 Transfer Learning and Model Modification

To train the model, ImageNet pre-trained weights are used for the initial weight configuration of the training process. This is because the weights are already learned from training on 1000 classes. For the initial training, the weights would remain unchanged except for the fully connected classification layer, where the weights are randomly initialized and updated during training the model. After evaluating the prediction results from the unchanged weights, the pretrained weights would be allowed to change for different layers; this verifies if optimizing the weights for this dataset improves classification compared to sticking with the pretrained weights. As discussed in tasks 3.1 and 3.2, the original classification layers are changed from 1000 outputs to 8 output layers for this dataset. This is the only modification that is made in order to preserve as much of the original ResNet18 architecture.

3.4 Model Input and Output Configuration

The table below shows the used modified ResNet18 architecture model on the input and output configuration.

Component	Configuration	Description
Input Image Size	224x224 pixels	The preprocessing would adjust the images to this size
Input Channels	3 Channels (RGB)	Red, blue, and green channels
Output Layer	8 output fully connected layers	Converts 512 feature maps into 8 output classes.
Activation Function	ReLU	Adds nonlinearity
Number Of Classes	8	Since there are CatEyes, DogEyes, CatFace, DogFace, CatMouth, DogMouth, CatNose, and DogNose classes.

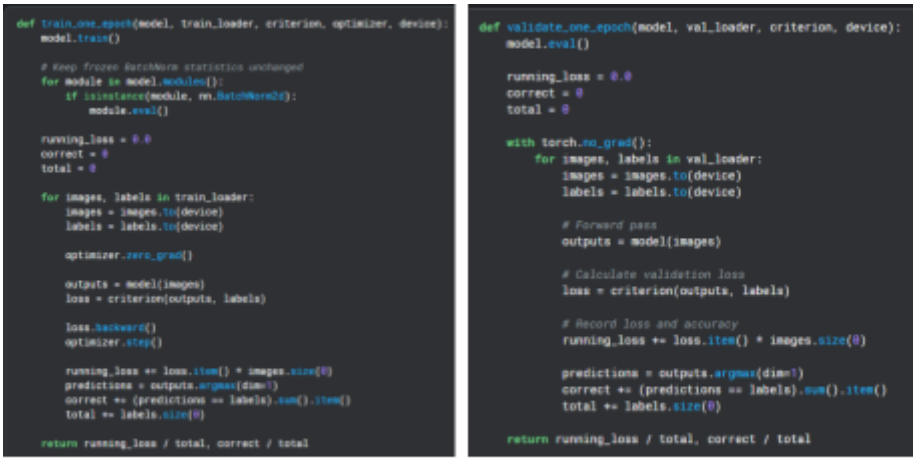
3.5 Training Hyperparameters

The table below shows the hyperparameters, methods, and values used for the initial training and validation of the model.

Hyperparameter	Selected Value	Justification
Loss Function	CrossEntropyLoss	This loss function method is used because this is a multiclass classification dataset. It allows the comparison of multiple class logits to check against the correct label class. This would produce a loss value that indicates how well the prediction aligns with the true label. This would help provide signals for updating weights.
Optimiser	Adam (Adaptive Moment Estimation)	This was selected because it utilizes previously calculated gradients and current gradients to update individual weights, since it helps decide the direction

		and size of what an individual weight normally receives to determine the next weight update. This improves efficiency and stability since it does not solely rely on the current gradient.
Learning Rate	0.001	This value is chosen because Adam and a 0.001 learning rate often pair together. If the learning rate is too high, it may cause an overly updated weight, which causes instability, while a low learning rate would be incredibly slow to train the model. This isn't necessarily an optimal choice, but it is a baseline until further training and validation loss is evaluated.
Batch Size	32	Determining the batch size is experimental, but since a Kaggle notebook would be used for training, a balance between training efficiency and memory consumption would be needed. Therefore, 32 is used as the starting point.
Number Of epochs	10	The number of epochs is set to 10 as the baseline value. This is experimental, as the validation results would need to be evaluated for overfitting or underfitting before changing the number of epochs.

3.6 Training Pipeline Implementation



```
def train_one_epoch(model, train_loader, criterion, optimizer, device):
    model.train()

    # Keep frozen BatchNorm statistics unchanged
    for module in model.modules():
        if isinstance(module, nn.BatchNorm2d):
            module.eval()

    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()

        outputs = model(images)
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)
        predictions = outputs.argmax(dim=1)
        correct += (predictions == labels).sum().item()
        total += labels.size(0)

    return running_loss / total, correct / total
```

```
def validate_one_epoch(model, val_loader, criterion, device):
    model.eval()

    running_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in val_loader:
            images = images.to(device)
            labels = labels.to(device)

            # Forward pass
            outputs = model(images)

            # Calculate validation loss
            loss = criterion(outputs, labels)

            # Record loss and accuracy
            running_loss += loss.item() * images.size(0)

            predictions = outputs.argmax(dim=1)
            correct += (predictions == labels).sum().item()
            total += labels.size(0)

    return running_loss / total, correct / total
```

Figure 3.6.1: Training and validate one epoch functions

The training pipeline was implemented in the Kaggle notebook using the accelerator GPU T4 x2. In the notebook, the preprocessing, training, and validation dataset were

loaded from task 2 through the created data_pipleine.py module that creates the dataloader for the training and validation. The ResNet18 architecture model was then loaded and configured with the task 3.5 specified hyperparameters.

Two functions have been created to facilitate the training, train one epoch and validate one epoch. The train one epoch function contains the process each batch of 32 images would pass through the ResNet18 model to generate the 8 classification values. These values are used to calculate loss from the implemented crossEntropyLoss. Then backpropagation was used to calculate the gradient in which the optimizer would update the trainable weights of the fully connected classification layer.

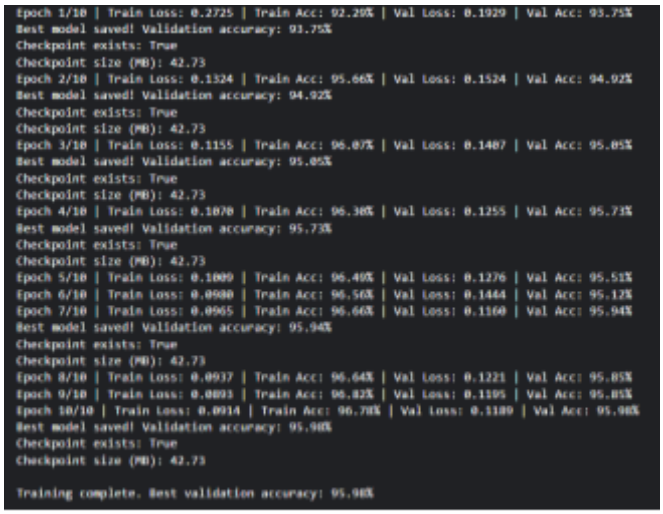


Figure 3.6.2: First model train result

The Validation process function, contains the process of validating one epoch from the trained weights of its corresponding training epoch run. The model gradient calculations were disabled to prevent weight updates.

The training and validation accuracy were recorded through the 10 epochs and the model train weights were saved whenever there is an improvement in validation accuracy so this can be reused for the Kaggle test prediction.

3.7 Kaggle Test Prediction Pipeline

To prepare a submission prediction for the test dataset, the ResNet18 model trained with the highest validation accuracy was saved and loaded to generate predictions on the unlabelled test dataset images. The model was placed into the same position as the validation step for 1 epoch, where no weights are changed because the gradient calculations are disabled. Each image is passed through the model to produce the 8 output logits, where the highest logit value is the predicted class. These values were converted into their corresponding class name to be prepared for storage in the submission.csv file.

In the submission.csv file, the predictions were then matched with their image ID and stored in the submission.csv file following the same format as the sample_submission.csv.

```
Total predictions: 29443
Total image IDs: 29443
First five predictions: ['DogEye', 'DogMouth', 'DogNose', 'DogFace', 'CatMouth']
```

Figure 3.7.1: First test dataset prediction

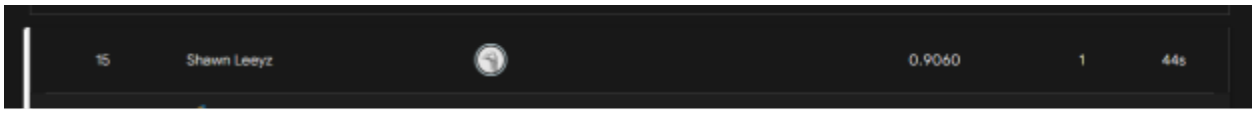


Figure 3.7.2: First submission uploaded to public Kaggle Leaderboard (16/10/2026)

4. Training Strategy and Overfitting/Underfitting Management

- [ ] Students must train their model and monitor its learning behaviour. Students are required to:
  - [ ] Train the selected model using the prepared training data.
  - [ ] Monitor training and validation accuracy and loss.
  - [ ] Identify signs of overfitting or underfitting.
  - [ ] Apply appropriate strategies to improve generalisation, such as data augmentation, dropout, weight decay, batch normalisation, early stopping, learning-rate scheduling, transfer learning, fine-tuning, or model adjustment.
  - [ ] Present and discuss learning curves for training and validation performance.

- [ ] Justify any changes made to the model or training strategy.

5. Model Evaluation and Kaggle Submission

- [ ] Students must evaluate their model and submit predictions to the Kaggle competition. Students are required to:
  - [ ] Evaluate the model on the validation set using suitable classification metrics.
  - [ ] Report validation accuracy and loss.
  - [ ] Include additional metrics where appropriate, such as precision, recall, F1-score, confusion matrix, or class-level accuracy.
  - [ ] Generate predictions for the Kaggle test images.
  - [ ] Prepare the submission file in the required Kaggle format.
  - [ ] Submit predictions to the Kaggle competition.
  - [ ] Report the Kaggle leaderboard score and include evidence of the submission.
  - [ ] Compare validation performance with Kaggle leaderboard performance.

6. Results Analysis and Improvement Strategies

- [ ] Students must critically analyse the results and discuss possible improvements. Students are required to:
  - [ ] Interpret the model performance using validation results and Kaggle score.
  - [ ] Analyse common misclassifications and difficult classes where possible.
  - [ ] Discuss possible reasons for performance limitations.
  - [ ] Evaluate the effectiveness of preprocessing, augmentation, model choice, and training strategy.
  - [ ] Compare attempted model versions or experimental settings where applicable.
  - [ ] Propose realistic strategies for improving performance, such as alternative architectures, hyperparameter tuning, improved augmentation, fine-tuning, ensemble methods, or better validation strategies.
  - [ ] Reflect on the strengths and weaknesses of the final solution.

7. Report Writing and Reproducibility

- [ ] Students must prepare a clear individual technical report and provide reproducible supporting files. Students are required to:
  - [ ] Write a structured final report of 3,000- 3,500 words, describing the complete workflow.
  - [ ] Include sufficient detail about the dataset, preprocessing, model, training strategy, evaluation, Kaggle results, analysis, and improvement strategies.

- [ ] Provide source code or a Jupyter Notebook with clear comments and instructions.
- [ ] Include the final Kaggle submission file.
- [ ] Include Kaggle username and leaderboard evidence.
- [ ] Reference all external libraries, models, papers, tutorials, or resources used.
- [ ] Ensure that the work is original and complies with academic integrity requirements.
- [ ] Submit a Turnitin similarity report with a similarity score not exceeding 20%.

References

Libraries

1. Matplotlib Development Team. (n.d.). Matplotlib documentation. https://matplotlib.org/stable/
2. The pandas development team. (n.d.). Pandas documentation. https://pandas.pydata.org/docs/
3. Pillow Contributors. (n.d.). Pillow documentation. https://pillow.readthedocs.io/en/stable/
4. PyTorch Contributors. (n.d.). PyTorch documentation. https://docs.pytorch.org/docs/stable/
5. Scikit-learn developers. (n.d.). Scikit-learn documentation. https://scikit-learn.org/stable
6. PyTorch Contributors. (n.d.). Torchvision documentation. https://docs.pytorch.org/vision/stable/
7. Python Software Foundation. (n.d.). hashlib—Secure hashes and message digests. https://docs.python.org/3/library/hashlib.html

Other References

1. Tanvir, I., Ruslan, A., & Solaiman, S. (2026). Comparative analysis of custom CNN architectures versus pre-trained models and transfer learning: A study on five Bangladesh datasets [Preprint]. arXiv. https://arxiv.org/abs/2601.04352
2. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 770–778). IEEE. https://doi.org/10.1109/CVPR.2016.90

3. Alrikabi, J. M., & Muhammad, S. S. (2024). Deep feature extraction with SVM classifier for brain tumor classification. *Journal of Education for Pure Science–University of Thi-Qar, 14*(3). https://doi.org/10.32792/jeps.v14i3.539